

examen 2017 / 18-2

asignatura	código	fecha	hora inicio
Análisis y diseño con patrones	05586	06/20/2018 09:00	

05.586 20 06 18 EX!
05586 20 06 18 EX

Pegue en este espacio una etiqueta identificativa
con su código personal
examen

Este enunciado corresponde también a las asignaturas siguientes:

- 06547 - Análisis y diseño con patrones

Ficha técnica del examen

- Comprueba que el código y el nombre de la asignatura corresponden a la asignatura matriculada.
- Sólo tienes que pegar una etiqueta de estudiante en el espacio correspondiente de esta hoja.
- No se pueden adjuntar hojas adicionales, ni realizar el examen en lápiz o rotulador grueso.
- Tiempo total: **2 horas** Valor de cada pregunta: **Indicado a la pregunta**
- En caso de que los estudiantes puedan consultar algún material durante el examen, cuáles son?

CAP En caso de poder utilizar calculadora, de qué tipo? **CAP**

- Si haya preguntas tipo test: descontar las respuestas erróneas? **SÍ** Cuánto? **indicado en la pregunta**

- Indicaciones específicas para la realización de este examen:

CAP

enunciados

examen 2017 / 18-2

asignatura	código	fecha	hora inicio
Análisis y diseño con patrones	05586	06/20/2018 09:00	

Pregunta 1 (10%)

Explica, en un máximo de cinco líneas, que es una construcción de software así como sus principales ventajas e inconvenientes.

solución:

Módulo 1. Apartado 3.1

Pregunta 2 (20%)

Explica brevemente los problemas que intentan resolver el patrones Adaptador y Asociación histórica.

solución:

Adapter: Módulo 2, apartado 6.8.1 Asociación histórica:
Módulo 2, apartado 3.1

Pregunta 3 (10%)

Responda si son ciertas o falsas las siguientes afirmaciones. No es necesario que justifique su respuesta. Cada una cuenta 2,5% si se acierta y descuenta 2,5% si se falla. Las respuestas en blanco no cuentan ni descuentan puntos. La nota mínima de esta pregunta será 0.

- a) El patrón observador es un patrón de análisis
(Falso. Módulo 2, apartado 6.6)
- b) Minimizar el acoplamiento significa que las clases no deben tener asociaciones
(Falso. Módulo 2, apartado 2.1)
- c) El principio de no-repetición dice que debemos evitar la duplicación de información y responsabilidades
(Cert. Módulo 2, apartado 2.4)
- d) El principio abierto-cerrado significa que el software debe poder incorporar nuevas funcionalidades sin necesidad de modificar el código existente
(Cert. Módulo 2, apartado 2.3)

examen 2017 / 18-2

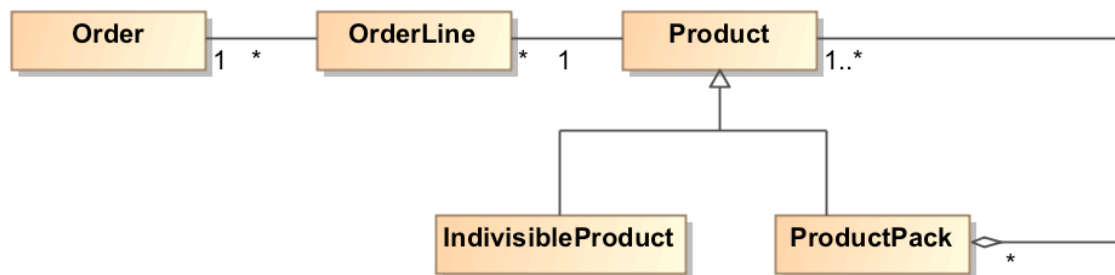
asignatura	código	fecha	hora inicio
Análisis y diseño con patrones	05586	06/20/2018 09:00	

Ejercicio 1 (30%)

Supongamos que estamos desarrollando un sistema de gestión de un comercio. En concreto, queremos guardar pedidos, donde cada pedido tiene una serie de líneas de comadre y cada línea de pedido tiene asociado un producto. A veces, los productos se venden en forma de pack. En este caso, el pack tiene igualmente un precio y puede estar asociado a una línea de pedido igual que un producto cualquiera. Con el fin de llevar un cierto control, sin embargo, es necesario saber, por un pack concreto, de qué productos está formado, ya que habrá que verificar aspectos relacionados con la política de precios y la gestión de stocks (funcionalidades que quedan fuera del alcance del ejercicio y que, por tanto, no tendremos en cuenta). Se pide el **diagrama estático de análisis** para registrar esta información. En caso de que hayas aplicado un patrón de análisis, indica qué patrón has aplicado. Haz los supuestos que creas necesarios y justifica tus decisiones.

solución:

Hemos aplicado el patrón "Objeto compuesto" (Composite) ya que queremos tratar los packs de productos y los productos indivisibles de manera uniforme.



examen 2017 / 18-2

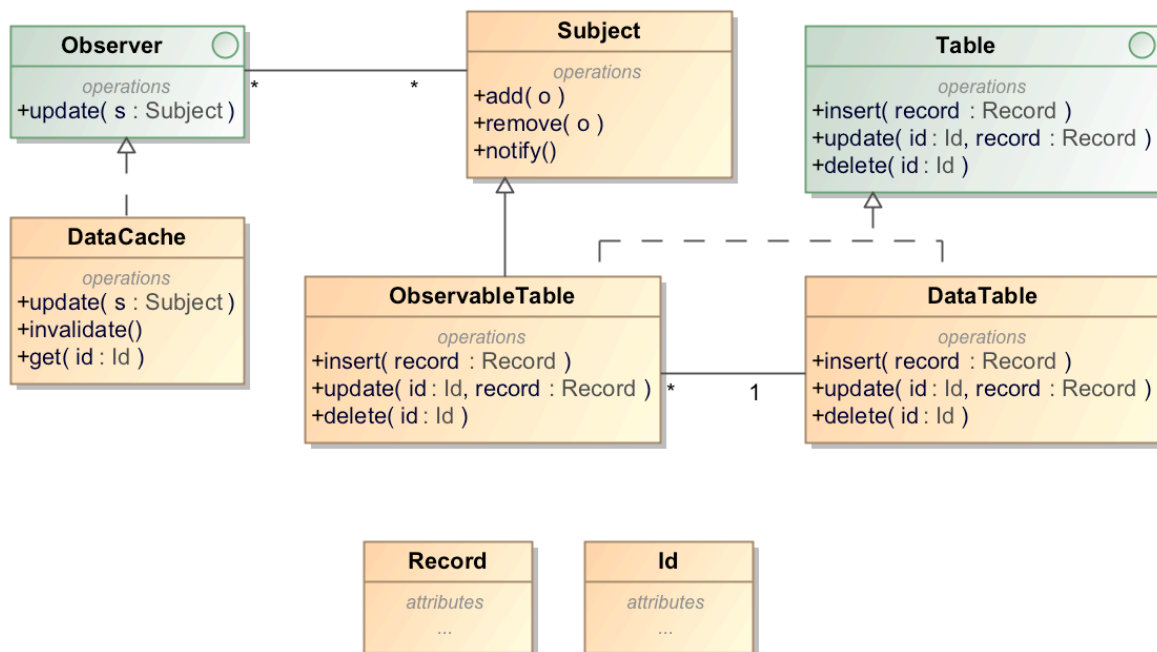
asignatura	código	fecha	hora inicio
Análisis y diseño con patrones	05586	06/20/2018 09:00	

Ejercicio 2 (30%)

Estamos desarrollando un software que tiene una serie de tablas de datos en memoria (*DataTable*) que alimentan unas cachés de datos (*DataCache*). Una caché de datos, necesita saber cuándo han cambiado los datos de la tabla para invalidar la caché (es decir, marcar los datos que había cachejados como inválidos). Partiendo del diseño original de la clase *DataTable*, hemos añadido el comportamiento que notifica los cambios sin modificar la clase original añadiendo la clase *ObservableTable* que implementa la misma interfaz que

DataTable de manera que sea transparente el hecho de haber añadido este comportamiento. En concreto, disponemos

de este diagrama de clases:



a) ¿Qué patrones se han aplicado en este diagrama de diseño? Indique qué clases del diagrama implementan los patrones detectados.

b) Proponer el diagrama de secuencia o escriba el pseudocódigo que muestre cómo se implementa la operación *insert* de la clase *ObservableTable* hasta llegar a la invocación de la operación *invalidate* de las instancias de *DataCache* que estén observando la tabla. Puede suponer que la operación *notify* de la clase *Subject* ya está implementada.

solución

a) Se han aplicado los patrones Observador (Observer) y Adaptador (Adapter).

las clases *ObservableTable* y *DataCache* implementan el patrón Observer. las clases *ObservableTable* y *DataTable* implementan el patrón Adapter.

b)

```

class ObservableTable {private DataTable
decoradas; public void insert (Record recuerdo)
{
    decorated.insert (recuerdo); notify
();
}

```

examen 2017 / 18-2

assignatura	código	fecha	<u>hora inicio</u>
Análisis y diseño con patrones	05586	<u>06/20/2018</u> 09:00	

```
}}
```

```
class DataCache () {  
    public void update (Subject s)  
    {this.invalidate (); }  
}
```